

Build a Training Management System (TMS) Backend

Build a production-ready **Training Management System (TMS)** backend using **ASP.NET Core Web API** on **.NET 10, C#, Entity Framework Core**, and **PostgreSQL**.

The system manages the complete lifecycle from:

Candidate → Application → Review → Exam/Interview → Selection → Cohort → Student → Training → Attendance/Assessment → Certificate

The first implementation priority is the **Application/Admission side**, but structure the project so the later Training, Attendance, Assessment, and Certificate modules can be added cleanly.

1. Technology Stack

Use:

- .NET 10
- ASP.NET Core Web API
- C#
- Entity Framework Core
- PostgreSQL
- Npgsql
- ASP.NET Core Identity or a clean JWT-based authentication implementation
- JWT Bearer authentication
- Role-based authorization
- FluentValidation where appropriate
- Swagger / OpenAPI
- Serilog for structured logging
- xUnit for unit/integration tests
- Docker / Docker Compose for local PostgreSQL
- RESTful JSON APIs
- UTC timestamps internally
- UUID/GUID primary keys
- PostgreSQL `timestampz`, `date`, `numeric`, etc.
- EF Core migrations

Do not use MongoDB or another NoSQL database.

The primary database must be PostgreSQL.

2. Architecture

Build this as a **modular monolith**, not microservices.

Use clean architecture / vertical-slice principles with clear separation between:

```
Tms.Domain
Tms.Application
Tms.Infrastructure
Tms.Api
Tms.Tests
```

Suggested structure:

```
src/
├── Tms.Domain/
│   ├── Common/
│   ├── Candidates/
│   ├── Applications/
│   ├── Admissions/
│   ├── Courses/
│   ├── Cohorts/
│   ├── Training/
│   ├── Students/
│   ├── Identity/
│   ├── Attendance/
│   ├── Assessments/
│   └── Certificates/
├── Tms.Application/
│   ├── Candidates/
│   ├── Applications/
│   ├── Admissions/
│   ├── Courses/
│   ├── Cohorts/
│   ├── Students/
│   ├── Training/
│   ├── Attendance/
│   ├── Assessments/
│   └── Certificates/
├── Tms.Infrastructure/
│   ├── Persistence/
│   └── Configurations/
```

```
| | | └─ Migrations/
| | |   └─ TmsDbContext.cs
| | └─ Identity/
| | └─ FileStorage/
| | └─ Email/
| | └─ Services/
| └─ Tms.Api/
|   └─ Controllers/
|   └─ Middleware/
|   └─ Extensions/
|   └─ Program.cs
|
└─ tests/
    └─ Tms.UnitTests/
    └─ Tms.IntegrationTests/
```

Keep domain logic out of controllers.

Controllers/endpoints should be thin.

Business rules should live in domain/application services/use cases.

3. Main Roles

The system will eventually have these roles:

```
Admin
Coordinator
Instructor
Student
```

For now, implement authorization infrastructure so that permissions can be expanded later.

Admin

Can:

- manage users
- assign roles
- manage coordinators
- manage instructors
- manage courses

- manage cohorts
- manage system configuration

Coordinator

Can:

- view applications
- review applications
- request additional information
- reject applications
- approve candidates for exam/interview
- schedule admission assessments
- enter exam/interview results
- filter candidates
- select candidates
- assign selected candidates to cohorts
- later manage sections and students

Instructor

Later can:

- view assigned sections
- take attendance
- enter training assessments
- view students in assigned sections

Student

Can:

- view own dashboard
- view profile
- view attendance
- view grades
- view course/cohort information
- later download certificate

4. Candidate and Application Model

A candidate is a person.

An application is a specific attempt by that candidate to join the TMS.

Do NOT merge Candidate and Application.

Relationship:

```
Candidate
├── EducationRecord[]
└── TrainingApplication[]
```

A candidate may potentially have multiple applications over time.

5. Candidate Entity

Create:

```
Candidate
```

Fields:

```
Id
CandidateNumber
FirstName
MiddleName
LastName
Email
Phone
DateOfBirth
Gender
Address
CreatedAt
UpdatedAt
```

Constraints:

- CandidateNumber unique
 - Email indexed
 - reasonable max lengths
 - email normalized/lowercased
 - required fields enforced
-

6. EducationRecord

Create:

EducationRecord

Fields:

Id
CandidateId
Institution
Qualification
FieldOfStudy
StartDate
EndDate
Grade
IsCompleted
CreatedAt

Relationship:

Candidate 1 — N EducationRecord

7. Application

Create:

TrainingApplication

Database table:

applications

Fields:

Id
ApplicationNumber

```
CandidateId
Status
SubmittedAt
CreatedAt
UpdatedAt
```

Application number must be human-readable, for example:

```
APP-2026-000001
APP-2026-000002
```

Do not use the GUID as the user-facing application number.

The GUID remains the technical primary key.

8. Application Status

Implement the following status values:

```
Draft
Submitted
UnderReview
AdditionalInformationRequired
ApprovedForAssessment
AssessmentScheduled
AssessmentCompleted
Passed
Failed
Accepted
Rejected
Withdrawn
```

Do not allow arbitrary status changes.

Implement valid status transitions.

For example:

```
Draft
  → Submitted
```

Submitted

→ UnderReview

UnderReview

→ AdditionalInformationRequired

→ ApprovedForAssessment

→ Rejected

AdditionalInformationRequired

→ UnderReview

ApprovedForAssessment

→ AssessmentScheduled

AssessmentScheduled

→ AssessmentCompleted

AssessmentCompleted

→ Passed

→ Failed

Passed

→ Accepted

The exact transition logic should be implemented in the application/domain layer rather than directly in controllers.

9. Application Status History

Every important status transition must be recorded.

Create:

ApplicationStatusHistory

Fields:

Id

ApplicationId

FromStatus

ToStatus

ChangedBy

Reason
ChangedAt

Example:

Draft → Submitted
Submitted → UnderReview
UnderReview → AdditionalInformationRequired
AdditionalInformationRequired → UnderReview
UnderReview → ApprovedForAssessment
ApprovedForAssessment → AssessmentScheduled
AssessmentScheduled → AssessmentCompleted
AssessmentCompleted → Passed
Passed → Accepted

This history must never be silently overwritten.

10. Application Documents

Candidates can upload:

- CV
- educational certificates
- transcript
- identification document
- other required documents

Create:

ApplicationDocument

Fields:

Id
ApplicationId
DocumentType
FileName
StorageKey
ContentType

```
FileSize
UploadedAt
```

Do NOT store large uploaded files directly inside PostgreSQL as `bytea` unless there is a strong reason.

Create an abstraction:

```
IFileStorage
```

so storage can initially be local filesystem and later be replaced by:

- S3
- Azure Blob Storage
- MinIO
- another object-storage service

The database stores metadata and storage key.

11. Application Questions

The TMS has short interview/application questions that candidates answer during application.

Create:

```
ApplicationQuestion
ApplicationAnswer
```

ApplicationQuestion:

```
Id
Question
QuestionType
IsRequired
DisplayOrder
IsActive
CreatedAt
UpdatedAt
```

ApplicationAnswer:

Id
ApplicationId
QuestionId
AnswerText
AnsweredAt

Initially support:

Text
LongText
YesNo
Number

Design the model so more types can be added later.

Add a unique constraint:

(ApplicationId, QuestionId)

12. Courses

Create:

Course

Fields:

Id
Code
Name
Description
IsActive
CreatedAt
UpdatedAt

Example:

```
WD
Web Development
```

Candidates select courses during application.

13. Application Course Selection

Create:

```
ApplicationCourse
```

This represents:

Candidate wants to study this course.

It does NOT represent actual enrollment.

Fields:

```
ApplicationId
CourseId
PreferenceOrder
SelectedAt
```

Use a composite primary key:

```
(ApplicationId, CourseId)
```

Example:

```
Application
|
├── Web Development
├── Data Analysis
└── Graphic Design
```

14. Cohorts / Batches

This is a very important part of the TMS.

A course can have many parallel cohorts/batches.

For example:

```
Web Development
├─ 2026-WEB-01
├─ 2026-WEB-02
├─ 2026-WEB-03
└─ 2026-WEB-04
```

Create:

Cohort

Fields:

```
Id
CourseId
Code
Name
Year
StartDate
EndDate
Capacity
Status
CreatedAt
UpdatedAt
```

Example:

```
Code: 2026-WEB-01
Name: Web Development Cohort 1
Year: 2026
Capacity: 30
```

Possible status:

Planned
OpenForSelection
SelectionClosed
Active
Completed
Cancelled

A course can have many cohorts.

Course 1 — N Cohorts

Do NOT put `CohortId` directly on `ApplicationCourse`.

A candidate applies for a course first.

The candidate is assigned to a cohort only after the selection/admission process.

15. Admission Review

Create:

ApplicationReview

Fields:

Id
ApplicationId
ReviewerId
Status
Comment
CreatedAt

Review status:

ApprovedForAssessment
Rejected
AdditionalInformationRequired

Keep review history.

Do not store only the latest review.

16. Additional Information Requests

A coordinator may discover that required information is missing.

Create:

```
AdditionalInformationRequest
```

Fields:

```
Id
ApplicationId
RequestedBy
Message
Status
RequestedAt
ResolvedAt
```

Status:

```
Open
Resolved
Cancelled
```

Example:

```
"Please upload your educational transcript."
```

Candidate can then provide the missing information and the application returns to review.

17. Admission Exam / Interview

Candidates who pass the initial review may be called for:

- Exam
- Interview

Create:

AdmissionAssessment

Fields:

Id
ApplicationId
Type
Status
ScheduledAt
CompletedAt
Score
MaxScore
Result
Notes
EvaluatorId

Assessment type:

Exam
Interview

Assessment status:

Pending
Scheduled
Completed
Cancelled

Assessment result:

Passed
Failed

The result is manually entered by an authorized coordinator/evaluator.

Do NOT build an automated online exam system yet.

18. Manual Candidate Selection

This is another critical business requirement.

After exam/interview results are entered, coordinators filter candidates according to their criteria and manually select the candidates who should become students.

Create:

ApplicationSelection

Fields:

Id
ApplicationId
CohortId
SelectedBy
Status
Reason
SelectedAt

Status:

Selected
Removed

This represents:

This application has been selected for this specific cohort.

It is NOT yet the same thing as student enrollment.

19. Selection Rules

When selecting a candidate:

1. Application must be valid.
2. Application must have passed required admission stages.
3. Candidate must have selected the relevant course.
4. Cohort must belong to that course.

5. Cohort must be open for selection.
6. Cohort capacity must not be exceeded.
7. Candidate should not already be selected into another cohort of the same course for the same intake.
8. Record who performed the selection.
9. Record timestamp.
10. Record optional reason.

Do not rely only on frontend validation.

Enforce important business rules in backend/application services.

20. Admission Decision

Keep final admission decision separate from candidate selection.

Create:

```
AdmissionDecision
```

Fields:

```
Id
ApplicationId
Decision
Reason
DecidedBy
DecidedAt
```

Decision:

```
Accepted
Rejected
```

Conceptually:

```
Assessment
  ↓
Selection
  ↓
Admission Decision
```

↓
Student Enrollment

Do not immediately create a student just because an exam was passed.

21. Candidate Selection UI/API Concept

The coordinator should eventually be able to request something like:

```
GET /api/v1/coordinator/applications/selection-candidates
```

with filters such as:

```
courseId  
cohortId  
applicationStatus  
examResult  
interviewResult  
minimumExamScore  
minimumInterviewScore  
submittedFrom  
submittedTo  
educationQualification
```

The response should include enough information for a coordinator to decide:

```
Candidate  
Application Number  
Course  
Exam Score  
Exam Result  
Interview Score  
Interview Result  
Application Status  
Documents Status
```

The coordinator can then select candidates.

Provide a separate command endpoint for selection rather than overloading a generic update endpoint.

22. Email Notification

When the candidate successfully submits an application, send an email.

Message should communicate approximately:

```
Application submitted successfully.  
  
Your application has been received by the TMS.  
  
Application ID: APP-2026-000001  
  
Please wait until the coordinators review your application and shortlist  
candidates.  
  
You can use your application ID to track your application status.
```

Create:

```
ISender
```

Do not hard-code SMTP implementation into the application layer.

Email should be sent as an application/infrastructure concern.

If asynchronous processing is implemented, use an outbox pattern or another reliable mechanism so an email failure does not silently lose the application submission.

23. Application Tracking

Candidate must be able to track an application using the application ID.

Example endpoint:

```
GET /api/v1/public/applications/{applicationNumber}/status
```

Return:

```
{
  "applicationNumber": "APP-2026-000001",
  "status": "UnderReview",
  "submittedAt": "...",
  "lastUpdatedAt": "...",
  "timeline": [
    {
      "status": "Submitted",
      "date": "..."
    },
    {
      "status": "UnderReview",
      "date": "..."
    }
  ]
}
```

Do not expose sensitive coordinator notes or internal information to the candidate.

24. API Organization

Use versioned APIs.

Example:

```
/api/v1/public/applications
/api/v1/public/courses

/api/v1/coordinator/applications
/api/v1/coordinator/applications/{id}/review
/api/v1/coordinator/applications/{id}/request-information
/api/v1/coordinator/applications/{id}/assessments
/api/v1/coordinator/applications/{id}/select

/api/v1/cohorts
/api/v1/courses

/api/v1/admin/users
/api/v1/admin/roles
```

Use appropriate HTTP methods:

```
GET
POST
PUT/PATCH where appropriate
DELETE where appropriate
```

Avoid generic endpoints such as:

```
POST /api/application/change-status
```

Prefer business-oriented commands:

```
POST /api/v1/coordinator/applications/{id}/start-review
POST /api/v1/coordinator/applications/{id}/request-information
POST /api/v1/coordinator/applications/{id}/approve-for-assessment
POST /api/v1/coordinator/applications/{id}/reject
POST /api/v1/coordinator/applications/{id}/assessments
POST /api/v1/coordinator/applications/{id}/select
```

25. Database Design

Use PostgreSQL.

Initial tables should include at least:

```
candidates
education_records

applications
application_documents

application_questions
application_answers

courses
application_courses

cohorts

application_reviews
additional_information_requests
```

```
admission_assessments
application_selections
admission_decisions

application_status_history
```

Use:

- UUID primary keys
- foreign keys
- unique constraints
- indexes
- check constraints where useful
- cascading deletes only where appropriate

Do not blindly use cascade delete everywhere.

Historical/audit information should be protected.

26. Important Database Constraints

Create at minimum:

```
candidates.candidate_number UNIQUE

applications.application_number UNIQUE

application_answers
    UNIQUE(application_id, question_id)

application_courses
    PRIMARY KEY(application_id, course_id)

cohorts.code UNIQUE

admission_decisions.application_id UNIQUE
```

Indexes:

```
candidates.email

applications.candidate_id
```

```
applications.status
applications.submitted_at

application_documents.application_id

application_answers.application_id
application_answers.question_id

application_courses.course_id

cohorts.course_id
cohorts.status
cohorts.year

application_reviews.application_id
application_reviews.reviewer_id

additional_information_requests.application_id
additional_information_requests.status

admission_assessments.application_id
admission_assessments.scheduled_at

application_selections.application_id
application_selections.cohort_id
application_selections.selected_at

application_status_history.application_id
application_status_history.changed_at
```

27. EF Core

Create:

```
TmsDbContext
```

Use separate `IEntityTypeConfiguration<T>` classes.

Example:

```
Persistence/  
└─ Configurations/
```



```
|— CandidateConfiguration.cs
|— EducationRecordConfiguration.cs
|— TrainingApplicationConfiguration.cs
|— ApplicationDocumentConfiguration.cs
|— ApplicationQuestionConfiguration.cs
|— ApplicationAnswerConfiguration.cs
|— CourseConfiguration.cs
|— ApplicationCourseConfiguration.cs
|— CohortConfiguration.cs
|— ApplicationReviewConfiguration.cs
|— AdditionalInformationRequestConfiguration.cs
|— AdmissionAssessmentConfiguration.cs
|— ApplicationSelectionConfiguration.cs
|— AdmissionDecisionConfiguration.cs
|— ApplicationStatusHistoryConfiguration.cs
```

Do not put all EF configuration inside `OnModelCreating`.

28. Enum Storage

Store application enums as strings in PostgreSQL rather than integer values.

For example:

```
Submitted
UnderReview
Rejected
Accepted
```

instead of:

```
1
2
3
4
```

This makes the database easier to inspect and reduces problems when enum ordering changes.

29. Security

Implement:

- JWT authentication
- password hashing
- role-based authorization
- secure password policy
- authorization policies
- user identity tracking
- validation of uploaded files
- file size limits
- allowed MIME types
- no direct exposure of internal storage paths
- no sensitive information in candidate tracking endpoint

Roles:

```
Admin
Coordinator
Instructor
Student
```

Use authorization attributes/policies rather than checking role names manually throughout controllers.

30. Auditability

For important coordinator/admin operations record:

```
Who
What
When
Which application
Previous state
New state
Reason
```

At minimum, application status transitions, reviews, selections, admission decisions, and assessment result entry must be traceable.

31. Error Handling

Implement global exception handling middleware.

Return consistent error responses using `ProblemDetails`.

Example:

```
{
  "type": "...",
  "title": "Application cannot be submitted",
  "status": 400,
  "detail": "Required document 'Transcript' is missing."
}
```

Do not expose stack traces in production.

32. Validation

Validate:

- required personal data
- email
- phone
- document types
- file size
- file type
- required application questions
- selected courses
- cohort/course relationship
- application status transitions
- assessment score ranges
- cohort capacity
- duplicate selections

For example:

```
Score >= 0
Score <= MaxScore
```

and:

```
Cohort.CourseId == ApplicationCourse.CourseId
```

must be validated.

33. Transactions

Operations such as application submission should be transactional.

Example:

```
Submit application
  ↓
Validate application
  ↓
Change status
  ↓
Create status history
  ↓
Save database
  ↓
Publish application-submitted event
  ↓
Send email
```

Do not leave the application in `Submitted` while failing to create its status history.

For selection:

```
Validate cohort
Validate capacity
Validate candidate
Create selection
Update relevant application state
Create history
Commit transaction
```

34. Do NOT overbuild the following yet

Do not spend the first implementation phase building:

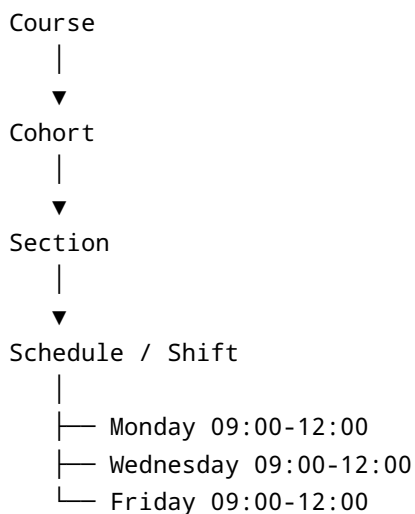
- online exam engine
- payment system
- SMS provider
- advanced analytics
- AI candidate scoring
- automated candidate ranking
- complex workflow engine
- microservices
- event streaming infrastructure
- mobile application
- sophisticated reporting
- certificate PDF generator

Keep extension points where appropriate, but build the core TMS first.

35. Future Training Module

Prepare the architecture for the next phase.

Eventually we will add:



Each section can have one or more instructors.

An instructor can teach many sections.

Eventually:

```
Section
|
├─ Instructor[]
├─ StudentEnrollment[]
└─ Schedule[]
```

Do not hardcode only four shifts into the database.

Represent schedules flexibly so the current four schedules can be configured as data.

Current examples:

```
Shift 1:
Monday, Wednesday, Friday
09:00 - 12:00

Shift 2:
Monday, Wednesday, Friday
16:00 - 19:00

Shift 3:
Tuesday, Thursday, Saturday
09:00 - 12:00

Shift 4:
Tuesday, Thursday, Saturday
16:00 - 19:00
```

36. Future Student Module

After admission:

```
Application
↓
ApplicationSelection
↓
AdmissionDecision = Accepted
↓
StudentEnrollment
```

↓
Student

Do not make `Candidate` itself become `Student` .

A person can transition from candidate to student while preserving their application history.

37. Future Attendance and Assessment

Eventually each student will have:

```
StudentEnrollment
├── AttendanceRecord[]
└── TrainingAssessment[]
```

Business completion rules:

```
Attendance >= 90%
AND
Assessment grade >= 50%
```

If both conditions are satisfied:

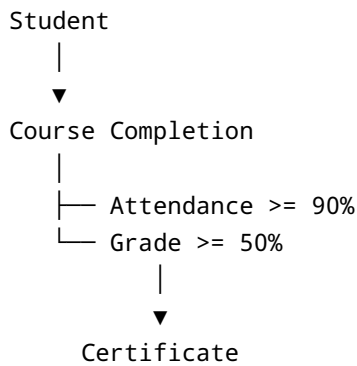
```
Course completion
↓
Certificate
```

These rules should be configurable eventually, but initially use:

```
Minimum attendance = 90%
Minimum passing grade = 50%
```

38. Certificate

Eventually:



Certificate should have a unique certificate number.

Do not implement this in the first admission slice unless necessary.

39. Testing Requirements

Create unit tests for domain rules.

At minimum test:

Application

Draft application can be submitted
Submitted application cannot be submitted again
Submitted application can enter review
Review can request additional information
Application can return to review
Only reviewed applications can be approved for assessment

Assessment

Assessment can be scheduled
Assessment can be completed
Score cannot exceed maximum score

Selection

Candidate can be selected into valid cohort
Candidate cannot be selected into wrong course cohort

Candidate cannot exceed cohort capacity
Duplicate selection is rejected

Admission

Passed application can be accepted
Failed application cannot be accepted

Integration tests

Test:

```
HTTP request
  ↓
API
  ↓
Application layer
  ↓
EF Core
  ↓
PostgreSQL
```

Use a real PostgreSQL test database/container for integration tests where practical.

40. Seed Data

Create development seed data for:

Roles

Admin
Coordinator
Instructor
Student

Courses

At least:

```
WEB - Web Development
DA  - Data Analysis
GD  - Graphic Design
```

Cohorts

Example:

```
2026-WEB-01
2026-WEB-02
2026-WEB-03

2026-DA-01
2026-DA-02
```

Application Questions

Examples:

```
Why do you want to join this program?

What is your previous experience?

What are your career goals?
```

Do not hard-code these values into business logic.

41. Swagger/OpenAPI

Configure Swagger and document:

- authentication
- endpoints
- request models
- response models
- validation errors
- authorization requirements

Make it easy to test the API from Swagger UI.

42. DTOs

Do not expose EF entities directly from API endpoints.

Use DTOs such as:

```
CreateApplicationRequest  
UpdateCandidateRequest  
EducationRecordRequest  
ApplicationDocumentResponse  
ApplicationResponse  
ApplicationSummaryResponse  
ApplicationStatusResponse  
SubmitApplicationResponse  
ReviewApplicationRequest  
RequestAdditionalInformationRequest  
ScheduleAssessmentRequest  
EnterAssessmentResultRequest  
SelectApplicationRequest  
CohortResponse  
CourseResponse
```

Separate request DTOs from response DTOs.

43. Application Submission Endpoint

The first complete vertical slice should support:

```
POST /api/v1/public/applications
```

to create a draft application.

Then:

```
PUT /api/v1/public/applications/{id}
```

or appropriate resource endpoints for updating the draft.

Then:

```
POST /api/v1/public/applications/{id}/submit
```

Submission should:

1. Validate candidate data.
2. Validate required education information.
3. Validate required questions.
4. Validate required documents.
5. Validate at least one course selection.
6. Change application status to `Submitted`.
7. Create status history.
8. Persist transactionally.
9. Return application number.
10. Trigger confirmation email.

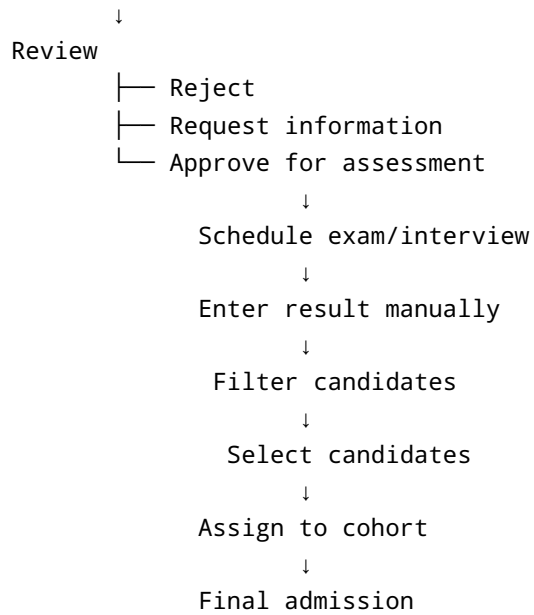
Response example:

```
{
  "applicationId": "guid",
  "applicationNumber": "APP-2026-000001",
  "status": "Submitted",
  "message": "Application submitted successfully."
}
```

44. Coordinator Workflow

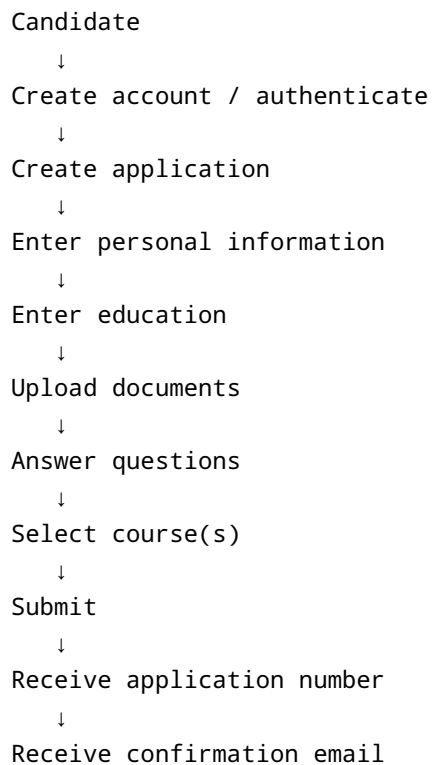
Implement the backend workflow:

```
Coordinator login
  ↓
List applications
  ↓
Filter/search
  ↓
Open candidate application
  ↓
View:
- personal details
- education
- documents
- answers
- selected courses
```



45. Candidate Workflow

Implement:



↓
Track status

46. Important Design Principle

Do NOT create the entire project as generic CRUD.

This is a business workflow application.

For example, don't build only:

```
PUT /applications/{id}
```

and allow arbitrary status updates.

Instead implement explicit use cases:

```
SubmitApplication  
StartApplicationReview  
RequestAdditionalInformation  
ApproveApplicationForAssessment  
RejectApplication  
ScheduleAdmissionAssessment  
EnterAssessmentResult  
SelectApplication  
AcceptApplication
```

This will make the business rules much safer.

47. Deliverables

Build the project, not just a design document.

The repository should contain:

```
.NET 10 solution  
PostgreSQL configuration  
EF Core DbContext  
EF Core migrations
```

```
Domain entities
Domain enums
Entity configurations
Application/use-case layer
DTOs
Validators
REST API
JWT authentication
Role authorization
Swagger
Global error handling
Structured logging
File storage abstraction
Email abstraction
Seed data
Unit tests
Integration tests
Docker Compose
README
```

The README must explain:

1. Prerequisites
2. How to configure PostgreSQL
3. Environment variables
4. How to run migrations
5. How to seed development data
6. How to start the API
7. Swagger URL
8. Default development accounts if seed accounts are created
9. Project architecture
10. Main application workflow

48. Implementation Order

Implement in this order.

Phase 1 — Foundation

```
Solution
Projects
Dependency injection
Configuration
Logging
```

Exception handling
Swagger
PostgreSQL
EF Core
Migrations

Phase 2 — Identity

Users
Roles
JWT
Authorization
Admin
Coordinator
Instructor
Student roles

Phase 3 — Candidate/Application

Candidate
EducationRecord
Application
ApplicationDocument
ApplicationQuestion
ApplicationAnswer
ApplicationCourse
ApplicationStatusHistory

Phase 4 — Courses/Cohorts

Course
Cohort

Phase 5 — Admission

ApplicationReview
AdditionalInformationRequest
AdmissionAssessment
ApplicationSelection
AdmissionDecision

Phase 6 — First complete vertical workflow

Make this work end-to-end:

```
Candidate
  ↓
Create application
  ↓
Upload documents
  ↓
Answer questions
  ↓
Select course
  ↓
Submit
  ↓
Application ID generated
  ↓
Email sent
  ↓
Coordinator reviews
  ↓
Assessment scheduled
  ↓
Result entered
  ↓
Coordinator filters
  ↓
Candidate selected
  ↓
Candidate assigned to cohort
```

Phase 7 — Training

After the above works, implement:

```
Section
Schedule
Shift
Instructor assignment
Student
Enrollment
```

Phase 8 — Training operations

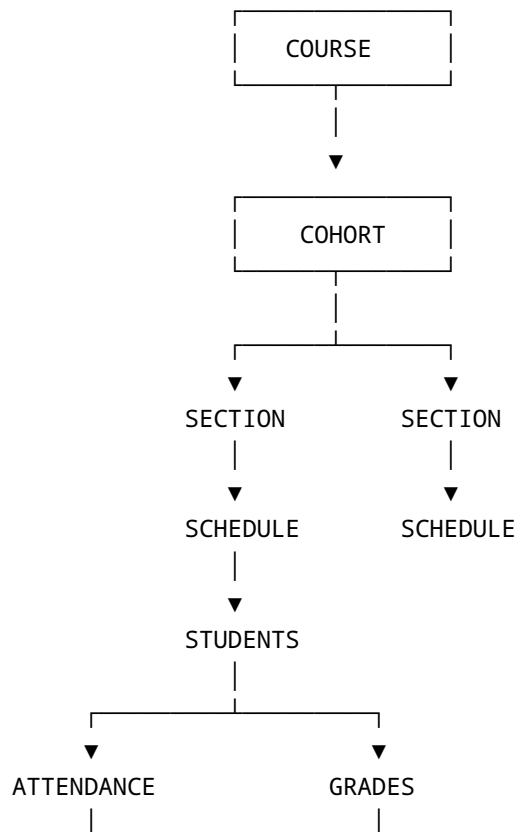
Attendance
Assessments
Grades
Student dashboard

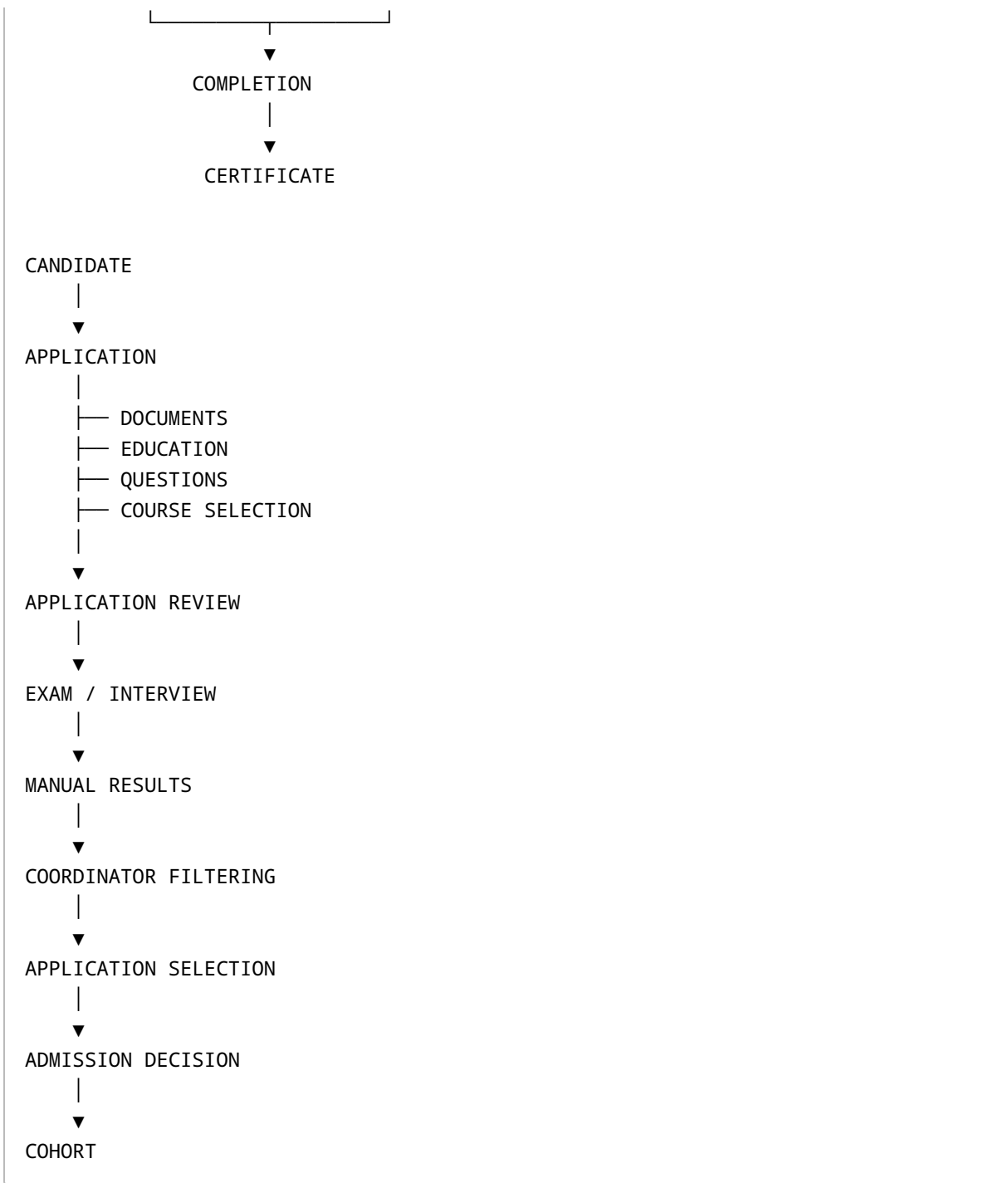
Phase 9 — Completion

90% attendance rule
50% grade passing rule
Course completion
Certificate

49. Final Architecture Goal

The final business model should evolve toward:





Build the backend incrementally around these bounded areas and preserve the relationships and business rules described above.

Most important: do not skip directly to CRUD generation. Build the domain model, database constraints, business rules, and application workflows first, then expose them through REST endpoints.

At every stage, keep the code compilable and runnable. After each major phase, run migrations and tests and fix compilation/test failures before proceeding.